Tradeflo AI Handover Notes

Tradeflo AI — Handover Notes

Architecture overview

Stack: Next.js 16 (App Router, Turbopack) + React 19 + TypeScript. Hosted on **Netlify**. Backend logic lives in Next.js API route handlers (`src/app/api/**/route.ts`) — there is no separate server.

Request flow / auth gate: `src/proxy.ts` (Next.js 16 `proxy.ts`, replaces `middleware.ts`) runs on every non-asset request and enforces, in order:

1. API routes pass through (no HTML redirects).
2. Logged-in users on `/login` or `/signup` → redirected to `next`.
3. Unauthenticated users on private paths → `/login`.
4. Incomplete onboarding → `/onboarding`.
5. No active subscription (and onboarding done) → `/billing`.
    - Sentry tunnel route `/monitoring` is excluded from the matcher.

Main app areas:

- **Auth** — `/login`, `/signup`, `/reset-password`, email confirm (`/api/auth/confirm`).
- **Onboarding** — `/onboarding` (business profile + work-log upload, or skip). Gated until complete.
- **Quote builder** — AI-generated quotes; draft → sent (immutable versions) → customer approval via public token link (`/approve/[token]`).
- **Billing** — `/billing` (Stripe Checkout / portal).
- **Admin** — `/admin/*` (materials catalog CRUD, catalog gaps, users). Access via `user_info.role = 'admin'`.

Code layout:

- `src/app/` — pages + API routes
- `src/lib/` — domain logic (`billing/`, `quote-generation/`, `quote-delivery/`, `onboarding/`, `data-retention/`, `observability/`, `supabase/`, `stripe/`, `admin/`)
- `src/components/` — UI
- `db/` — SQL migrations (apply manually in Supabase)
- `netlify/functions/` — scheduled cron function

Database overview

Supabase Postgres. All user data is scoped by `user_id` with **Row-Level Security** (users see only their own rows). Service-role/admin client bypasses RLS for webhooks and cron. Migrations in `db/` are applied **manually** in the Supabase SQL editor (run in order; all idempotent).

Tables:

Table	Purpose
`user_info`	One row per auth user (created by `handle_new_user` trigger on signup). Profile + onboarding flags + billing mirror (`stripe_customer_id` , `stripe_subscription_id` , `billing_subscription_status` , grace/read-only, `data_retention_purge_after_at`).
`quotes`	Per-user quote thread; `status` mirrors head version.
`quote_versions`	Immutable version payloads (JSON); holds approval token + status (`draft` / `sent` / `approved` / `changes_requested`).
`work_logs`	Uploaded work-history files + extracted text (AI input). Files in private Storage bucket `work-logs/{userId}/` .
`materials_catalog`	Admin-maintained reference retail prices per trade (read-only to contractors).
`materials_catalog_gaps`	Logged when AI returns an `estimated` material (no catalog match); admin reads via service role for catalog expansion.

Migration files (apply in order): `user_info.sql` → `user_info_roles.sql` → `onboarding.sql` → `quotes.sql` → `quotes_m2.sql` → `approval_feedback.sql` → `milestone3_user_info.sql` → `labour_rates.sql` → `quote_ai_rate_limit.sql` → `materials_catalog.sql` → `materials_catalog_gaps.sql` → `data_retention_purge.sql` .

Storage: private bucket `work-logs` (RLS by `{userId}/` path prefix).

Integrations overview

Service	Use	Key env vars	Code
Supabase	Auth, Postgres, Storage	`NEXT_PUBLIC_SUPABASE_URL` , `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` , `SUPABASE_SERVICE_ROLE_KEY`	`src/lib/supabase/*`

Service	Use	Key env vars	Code
Anthropic (Claude)	AI quote generation from text + site photos (via Vercel AI SDK `@ai-sdk/anthropic`)	`ANTHROPIC_API_KEY`	`src/lib/quote-generation/*`
Stripe	Subscription billing — 14-day trial, Checkout, billing portal, webhooks; GST/HST via Stripe Tax	`STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID`, `STRIPE_WEBHOOK_SECRET`	`src/lib/billing/*`, `src/lib/stripe/*`, `/api/billing/*`, `/api/webhooks/stripe`
Resend	Quote delivery via email	`RESEND_API_KEY`, `RESEND_FROM`	`src/lib/quote-delivery/resend-send.ts`
Twilio	Quote delivery via SMS	`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_MESSAGING_SERVICE_SID` / `TWILIO_PHONE_NUMBER`	`src/lib/quote-delivery/twilio-send.ts`
Sentry	Error tracking + structured logs; browser envelopes tunneled via `/monitoring`	`NEXT_PUBLIC_SENTRY_DSN`	`src/lib/observability/*`, `next.config.ts`
Netlify	Hosting + scheduled cron	`NEXT_PUBLIC_BASE_URL`, `DATA_RETENTION_CRON_SECRET`	`netlify/functions/`, `netlify.toml`

Key flows:

- **Billing:** Checkout resolves/creates a single Stripe customer per user (deduped by `metadata.supabase_user_id`), one subscription per user. Webhooks (`checkout.session.completed`, `customer.subscription.*`, `invoice.*`) sync status into `user_info`. Webhook secret differs local (Stripe CLI) vs production (Dashboard endpoint).
- **Quote delivery:** email (Resend) / SMS (Twilio) send a public approval link; customer approves or requests changes via token (no login).
- **Data retention:** on `customer.subscription.deleted`, `user_info.data_retention_purge_after_at` = now + 90 days. A daily Netlify scheduled function calls `/api/cron/data-retention-purge` (Bearer `DATA_RETENTION_CRON_SECRET`) to delete work logs, quotes, and storage for due users.